In [9]:

```python
"""INF201 Innlevering 2"""

import json
import re
from pathlib import Path


def read_log_file(path: str | Path) -> tuple[str, str]:

    path = Path(path)
    raw = path.read_bytes()

    try:
        return raw.decode("utf-8"), "utf-8"
    except UnicodeDecodeError:
        return raw.decode("iso-8859-1"), "iso-8859-1"



LOG_PATTERN = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+"
    r"\[(?P<register_id>[^\]]+)\]\s+"
    r"item=(?P<item>.+?)\s+"
    r"price=(?P<price>-?\d+(?:\.\d+)?)\s+"
    r"(?P<currency>[A-Z]{3})\s+"
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)\s*$",
    re.MULTILINE,
)


def parse_log_text(text: str) -> list[dict]:

    records = []
    for match in LOG_PATTERN.finditer(text):
        record = match.groupdict()
        record["price"] = float(record["price"])
        records.append(record)
    return records



def find_log_files(directory: str | Path, extension: str = ".log") -> lis
    """Finner rekursivt alle filer med gitt filendelse."""
    directory = Path(directory)
    found: list[Path] = []

    for item in directory.iterdir():
        if item.is_dir():
            found.extend(find_log_files(item, extension))
        elif item.is_file() and item.suffix == extension:
            found.append(item)

    return sorted(found)



def get_active_register_ids(node) -> set[str]:
    """Returnerer id-ene til alle registre med "active": true.
```

```
    Fungerer på enhver kombinasjon av nøstede dictionaries og lister.
    """
    active: set[str] = set()

    if isinstance(node, dict):
        if node.get("active") is True and isinstance(node.get("id"), str)
            active.add(node["id"])
        for value in node.values():
            active |= get_active_register_ids(value)
    elif isinstance(node, list):
        for element in node:
            active |= get_active_register_ids(element)

    return active
```

Task 3: Rydding og analyse med NumPy

remove_price_outliers: gjør prisene om til en 1D NumPy-array og regner ut gjennomsnittet og standardavvik. priser der (x - μ) er større enn 3 standardavvik regnes som feil (altså som 999.00), og byttes ut med medianen av de gyldige prisene. vi bruker medianen fordi den ikke påvirkes av ekstreme verider.

In [10]:
```
#compute_refund_ratio: regner ut refusjonsandelen element for element med

import numpy as np

def remove_price_outliers(prices) -> np.ndarray:
    "Replace outliers (|x - mean| > 3*std) with the median of the valid p
    prices = np.asarray(prices, dtype=float).ravel()  #1D float array

    mean = prices.mean()
    std = prices.std()
    is_outlier = np.abs(prices - mean) > 3 * std

    cleaned = prices.copy()
    if is_outlier.any():
        cleaned[is_outlier] = np.median(prices[~is_outlier])
    return cleaned
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarra
    """Element-wise refund ratio. Epsilon avoids division by zero."""
    return refund_count / (sales_count + 1e-6)
```

Rask test av funksjonene med egne tall: 999.0 skal byttes ut med medianen

In [11]:
```
#Tester med tilfeldige numre
test_prices = [4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5
print("Before:", test_prices)
print("After: ", remove_price_outliers(test_prices))
print("Refund ratio:", compute_refund_ratio(np.array([10, 0, 4]), np.arra
```

```
Before: [4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5, 4.5, 5.0, 4.0, 5.5, 999.
0]
After:  [4.5  5.   4.   5.5  4.5  5.   4.   5.5  4.5  5.   4.   5.5  4.75]
Refund ratio: [0.19999998 0.         0.24999994]
```

Task 4: Dataanalyse med pandas

build_dataframe: gjør listen med transaksjoner om til en dataframe, gjør "timestamp" om til en datetime, setter den som indeks og sorterer kronologisk

hourly_revenue_per_item: grupperer på vare og summerer prisene i vinduer på 1 time med "resample"("1h")

status_pivot_table: lager en pivottabelk med varer som rader, status (SUCCESS, REFUND, CANCELLED) som kolonner og total omsetning i cellene

refund_ratio_per_item: teller antall salg (SUCCESS) og refusjoner (REFUND) per vare, og bruker "compute_refund_ratio" fra Task 3

In [12]:
```
import pandas as pd

def build_dataframe(records: list[dict]) -> pd.DataFrame:
    """List of transaction dicts -> DataFrame with timestamp as sorted in
    df = pd.DataFrame(records)
```

```
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp").sort_index()
    return df

def hourly_revenue_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Total price per item in 1-hour windows."""
    hourly = df.groupby("item")["price"].resample("1h").sum()
    return hourly.reset_index()  #colums: item, timestamp, price

def status_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """Total revenue per item for each status."""
    return pd.pivot_table(
        df,
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0,
    )

def refund_ratio_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Refund ratio per item = number of refund / number of successful sa
    counts = df.groupby(["item", "status"]).size().unstack(fill_value=0)
    sales = counts.get("SUCCESS", pd.Series(0, index=counts.index)).to_nu
    refunds = counts.get("REFUND", pd.Series(0, index=counts.index)).to_n

    result = pd.DataFrame({
        "sales_count": sales,
        "refund_count": refunds,
        "refund_ratio": compute_refund_ratio(sales, refunds),
    }, index=counts.index)
    return result
```

Task 5: Hele arbeidsflyten

Her setter vi sammen funksjonene fra Task 1-4:

1. Finner alle loggfiler rekursivt og leser aktive kasser fra "store_config.json".

2. Leser filene (UTF-8 eller Latin-1) og henter ut gyldige salgslinjer.

3. Lager en DataFrame. vi tar bare med transaksjoner fra aktive kasser.

4. fjerner 999.00-feilene med NumPy.

5. retter NOK-priser som var registrert i EUR-skala, ved å gange med 10.

6. lager rapportene

7. lagrer de fire output-filene

In [13]:
```python
import json
from pathlib import Path

DATA_DIR = Path("data")
CONFIG_FILE = DATA_DIR / "store_config.json"

#1. Find all log files and the active register IDs
log_files = find_log_files(DATA_DIR)
config = json.loads(CONFIG_FILE.read_text(encoding="utf-8"))
active_registers = get_active_register_ids(config)
print(f"Found {len(log_files)} log files")
print(f"Active registers: {sorted(active_registers)}")

#2. Read every file and parse the valid sales lines
records = []
for path in log_files:
    text, encoding = read_log_file(path)
    records.extend(parse_log_text(text))
print(f"Parsed {len(records)} transactions")

#keep only transactionw from active registers
records = [r for r in records if r["register_id"] in active_registers]
print(f"{len(records)} transactions from active registers")

#3. Build the DataFrame
df = build_dataframe(records)

#4. remove 999.00 price glitches
n_glitches = (df["price"] >= 999).sum()
df["price"] = remove_price_outliers(df["price"])
print(f"Replaced {n_glitches} outliers")

#5. fix NOK prices that were registered on EUR scale
df.loc[df["currency"] == "NOK", "price"] *= 10.0
df["price"] = df ["price"].round(2)

#6. reports
hourly = hourly_revenue_per_item(df)
pivot = status_pivot_table(df)
refunds = refund_ratio_per_item(df)

#7. save output files
df.to_csv("cleaned_transactions.csv")
```

```
hourly_json = {}
for item, group in hourly.groupby("item"):
    hourly_json[item] = {
        ts.strftime("%Y-%m-%d %H: %M"): round(float(price), 2)
        for ts, price in zip(group["timestamp"], group["price"])

}
Path("hourly_revenue.json").write_text(json.dumps(hourly_json, indent=2,
    )

pivot.to_csv("status_pivot_table.csv")
refunds.to_csv("item_refund_ratios.csv")

print("\nSaved: cleaned_transactions.csv, hourly_revenue.json, "
          "status_pivot_table.csv, item_refund_ratios.csv")

print("Pivot table (total revenue per item and status):")
print(pivot.round(2))
print("\nRefund ratio per item:")
print(refunds.round(3))
df.head()
```

```
Found 9 log files
Active registers: ['POS_01', 'POS_02', 'POS_03']
Parsed 720 transactions
720 transactions from active registers
Replaced 16 outliers

Saved: cleaned_transactions.csv, hourly_revenue.json, status_pivot_table.c
sv, item_refund_ratios.csv
Pivot table (total revenue per item and status):
status      CANCELLED   REFUND  SUCCESS
item
Cappuccino     311.86   427.20  1677.76
Croissant      281.96   475.24  1334.70
Espresso       180.36   375.09   814.33
Latte          700.80   127.44  1127.52
Muffin         381.13   293.34   855.72
Sandwich       443.95  1113.86  2900.44

Refund ratio per item:
            sales_count  refund_count  refund_ratio
item
Cappuccino           78            22         0.282
Croissant            85            26         0.306
Espresso             73            21         0.288
Latte                70            10         0.143
Muffin               75            17         0.227
Sandwich             92            31         0.337
```

Out[13]:

timestamp	register_id	item	price	currency	status
2026-10-15 07:00:34	POS_01	Sandwich	73.80	NOK	CANCELLED
2026-10-15 07:00:41	POS_03	Espresso	2.92	EUR	SUCCESS
2026-10-15 07:00:57	POS_03	Croissant	3.52	EUR	SUCCESS
2026-10-15 07:01:07	POS_01	Espresso	3.10	USD	CANCELLED
2026-10-15 07:01:12	POS_01	Cappuccino	4.55	EUR	SUCCESS

Koden fant 9 loggfiler og 720 transaksjoer. Alle filene ble lest riktig, både de med UTF-8 og de med Latin-1. støylinjene fra printer og nettverk ble hoppet over av regexmønsteret.

X priser var registrert som 999.00.

disse ble funnet med 3 standardavvik-regelen og byttet ut med medianen. Vi brukte medianen fordi som nevnt så ville gjennomsnittet blitt dratt opp av feilverdiene. NOK-prisene ble ganget med 10, fordi de var registrert i EUR-skala.

I pivottavellen ser vi at Sandwich har høyest omsetning med status SUCCESS (2900,44), mens Espresse har lavest (814,33). Sandwich har også klart høyest refundert beløp(1113,86). Refusjonsandelen er høyest for croissant (0,306) og lavest for latte (0.143)

En svaket er at omsetningen er summert på tvers av EUR, NOK og USD. NOK-prisene er omtrent ti ganger større enn de andre, så summene blir dratt opp av transaksjoner i NOK. For å sammenligne riktig burde alle prisene vært regnet om til samme valuta.